

Assembler

Основные понятия и инструкции

Подготовил
Старший преподаватель
МИЭМ НИУ ВШЭ
Морозов Владимир Игоревич

Регистры

Основные регистры

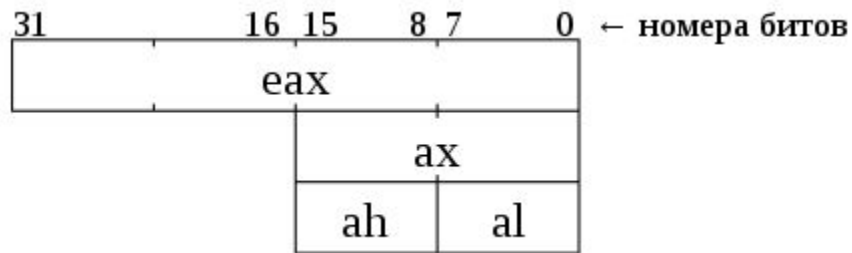
- **eax**: Accumulator register — аккумулятор, применяется для хранения результатов промежуточных вычислений.
- **ebx**: Base register — базовый регистр, применяется для хранения адреса (указателя) на некоторый объект в памяти.
- **ecx**: Counter register — счетчик, его неявно используют некоторые команды для организации циклов (см. loop).
- **edx**: Data register — регистр данных, используется для хранения результатов промежуточных вычислений и ввода-вывода.

Основные регистры

- **esp**: Stack pointer register — указатель стека. Содержит адрес вершины стека.
- **ebp**: Base pointer register — указатель базы кадра стека (англ. stack frame). Предназначен для организации произвольного доступа к данным внутри стека.
- **esi**: Source index register — индекс источника, в цепочечных операциях содержит указатель на текущий элемент-источник.
- **edi**: Destination index register — индекс приёмника, в цепочечных операциях содержит указатель на текущий элемент-приёмник.

Обращение к части регистра

В ассемблере предоставляется возможность обращаться не только к регистру целиком, но также к его частям (половинам, четвертям), как к отдельным регистрам, не затрагивая остальные части.



Флаги

Флаги — отдельные биты регистра флагов, к каждому из которых можно обратиться. Каждый бит содержит информацию о каком-то аспекте текущего состояния программы.

- **cf**: carry flag, флаг переноса:
 - 1 — во время арифметической операции был произведён перенос из старшего бита результата;
 - 0 — переноса не было;
- **zf**: zero flag, флаг нуля:
 - 1 — результат последней операции нулевой;
 - 0 — результат последней операции ненулевой;

Флаги

- **of**: overflow flag, флаг переполнения:
 - 1 — во время арифметической операции произошёл перенос в/из старшего (знакового) бита результата;
 - 0 — переноса не было;
- **df**: direction flag, флаг направления. Указывает направление просмотра в строковых (цепочечных) операциях:
 - 1 — направление «назад», от старших адресов к младшим;
 - 0 — направление «вперёд», от младших адресов к старшим.

Синтаксис

Синтаксисы ассемблера

Есть два основных вида синтаксиса ассемблера:

- Синтаксис Intel
- Синтаксис AT&T

Первый, как правило, используется в ОС семейств DOS и Windows, второй – в UNIX-подобных системах. В данной презентации рассматривается синтаксис AT&T.

Синтаксис ассемблера

- Программа на ассемблере, написанная с синтаксисом AT&T, содержит, как правило, две основные **секции** (вообще их может быть больше):
 - Секция данных (пишется `.data`) содержит объявление переменных
 - Секция кода (пишется `.text`) содержит непосредственно код программы
- **Метки** – некоторые константы, содержащие адрес в программе, на котором они установлены. Пишется **имя_метки**: (двоеточие важно). Метки позволяют отмечать места в коде или данные, к которым необходимо вернуться в ходе выполнения (например, точки входа в цикл или переменные)

Синтаксис ассемблера

- **Команды** на ассемблере записываются следующим образом:
`название_команды арг1, арг2`
никаких символов в конце не добавляется. Аргументы вписываются в нужном количестве при необходимости (есть команды без аргументов).
- Существует ряд команд, которые используют только конкретные заранее определённые регистры для своей работы. Изменить это поведение невозможно. Эти регистры необходимо просто знать.
- Команды выполняются последовательно от строки к строке **независимо от меток**. Изменить последовательность можно только специальной командой.

Синтаксис ассемблера

- В синтаксисе AT&T к команде, работающей с данными, справа приписывается буква, указывающая на размер данных:
 - **b** (от англ. byte) — 1 байт,
 - **w** (от англ. word) — 2 байта,
 - **l** (от англ. long) — 4 байта,
 - **q** (от англ. quad) — 8 байт.
- Например, чтобы поместить число **42** в регистр **a1**, имеющий размер 1 байт, к команде перемещения **mov** добавляется буква **b**. Получается:
movb \$42, %a1

Синтаксис ассемблера

- Как видно из примера на предыдущем слайде, для работы с регистрами перед именем регистра необходимо ставить знак %, а для работы с числами перед ними следует ставить знак \$ (число без символа \$ будет распознано как адрес)

Адресация

- В ассемблере, как и в Си, есть операторы получения адреса и разадресации.
- Оператор получения адреса (пишется `$`) позволяет получить адрес метки (полезно, если метка стоит на данных)
- Оператор разадресации (пишется **(операнд)**) позволяет получить данные, хранящиеся по адресу, указанному в операнде
- Например, чтобы поместить в регистр **eax** данные, находящиеся по адресу, записанному в регистр **ebx**, нужно сделать:

```
mov (%ebx) , %eax
```

Основные команды

Перемещение

- `mov` – команда, позволяющая выполнять перемещение данных (похоже на присваивание в Си)
- Принимает два операнда: источник и приёмник (для синтаксиса AT&T порядок операндов именно такой, для Intel – наоборот)
- Примеры:

`movl %eax, %ebx` # Кладёт содержимое регистра `eax` в регистр `ebx`

`movl $55, %eax` # Кладёт число 55 в регистр `eax`

Условия

- В ассемблере не существует привычного нам условного оператора (наподобие `if`)
- Условные переходы осуществляются в ассемблере в два этапа:
 - Проверка условия и установление флагов
 - Выполнение перехода в зависимости от значений флагов
- Для первого этапа, как правило, используется команда `cmp`. Она сравнивает два операнда, как числа, и выставляет флаги в качестве результата.
- Для второго этапа используется семейство команд перехода `jump`.

Команды перехода

- Название команд перехода начинается с буквы j. Далее идёт последовательность букв, означающая условие, по которому осуществляется переход.
- В качестве аргумента команда принимает адрес (метку), по которому нужно осуществить переход в случае выполнения условия

Мнемоника	Английское слово	Смысл	Тип операндов
e	equal	равенство	любые
n	not	инверсия условия	любые
g	greater	больше	со знаком
l	less	меньше	со знаком
a	above	больше	без знака
b	below	меньше	без знака

Команды перехода

- Например, чтобы сравнить значения регистров `eax` и `ebx` и осуществить переход на метку `m` в случае их равенства, нужно сделать:

```
cmp %eax, %ebx
```

```
je m
```

- `je` можно прочесть как `jump if equal`. Чтобы осуществить переход при неравенстве, можно использовать `jne` (`jump if not equal`)
- Если условие не выполняется, программа переходит к следующей за `jump`-командой строке
- Чтобы выполнить **безусловный** переход к метке, нужно использовать `jmp`

Циклы

Циклы

- Для циклического выполнения команд используется команда `loop`
- Эта команда **неявно использует регистр `ecx` в качестве счётчика** цикла
- Чтобы организовать цикл, необходимо:
 - В регистр `ecx` поместить необходимое количество итераций
 - Поставить метку в точке входа в цикл
 - В конце тела цикла вписать команду `loop` с меткой входа в цикл
- Каждый раз, доходя до команды `loop`, программа будет уменьшать значение в `ecx` и переходить к нужной метке. Когда значение в `ecx` дойдёт до 0, переход не произойдёт. Программа перейдёт к следующей строке.

Циклы

- Пример:

```
movl $10, %ecx
```

```
cycle:
```

```
    // Какие-то действия, тело цикла
```

```
    // Ещё действия
```

```
    // ...
```

```
loop cycle
```

- В данном примере тело цикла будет выполнено 10 раз

Вызов подпрограмм

Вызов подпрограмм

- Подпрограммы в ассемблере оформляются как блоки кода, отмеченные отдельной меткой, никакого дополнительного синтаксиса нет
- Чтобы перейти к коду подпрограммы, необходимо выполнить команду `call`, принимающую адрес подпрограммы (метку) в качестве аргумента
- Чтобы вернуться из подпрограммы к вызывающему коду, нужно выполнить `ret`
- Грубо говоря, `call` выполняет безусловный переход к указанной метке и “запоминает”, откуда он перешёл. `ret` возвращается к “запомненному” месту.

Вызов подпрограмм

Пример:

```
// Основной код
// ...
// ...

call f
//...
//...

f:
// Код подпрограммы
// ...
// ...
ret
```

The diagram illustrates the control flow of a program call. A black arrow originates from the `call f` instruction and points to the label `f:`, indicating the transfer of control to the subprogram. A red arrow originates from the `ret` instruction at the end of the subprogram and points back to the line immediately following the `call f` instruction, representing the return of control to the caller.

Работа с данными

Объявление данных

- Данные (по смыслу – примерно аналоги переменных) объявляются в секции `.data`
- Оъявление производится следующим образом:
название_переменной: .тип значение
- Типы данных:
 - `byte`
 - `short`
 - `long`
 - `quad`
 - `string`

Объявление данных

- Например, объявить “переменную” типа `long` со значением 0 можно так:

```
var: .long 0x0
```

а массив того же типа, заполненный числами по возрастанию, так:

```
arr: .long 0x0, 0x1, 0x2, 0x3, 0x4, 0x5
```

- Не инициализируемые заранее данные объявляются с помощью типа `.space`, после которого, вместо значений, идёт кол-во выделяемых под данные байт памяти, например:

```
some_data: .space 128
```

выделит под переменную `some_data` 128 байт.

Компиляция и запуск программы

Компиляция и запуск программы

- В Linux для компиляции программ на ассемблере используется компилятор `gcc`.
- Чтобы скомпилировать программу, необходимо создать в ней метку `main`, которая будет являться точкой входа.
- Далее следует вызвать
`gcc название_файла.s -no-pie -o название_исп_файла -g`
- Если сообщений об ошибках не поступило, компиляция прошла успешно, можно запустить программу на исполнение, выполнив:
`./название_исп_файла`

Отладка

Зачем нужен отладчик

- Вывод данных из программы на ассемблере – отдельная и достаточно объёмная задача (можно взять как доп. задание).
- Чтобы продемонстрировать, что программа отработала успешно, будет использоваться средство отладки gdb.
- Также очень полезно использовать gdb по его прямому назначению – для отладки программы и поиска ошибок в процессе разработки.

Как использовать gdb

Чтобы продемонстрировать работу программы, нужно:

- Выполнить:
`gdb название_исп_файла`
- Установить точку останова на точке входа или нужной строке
- Запустить выполнение программы
- При необходимости выполнить код по шагам
- Показать значения регистров и данных с помощью средств gdb

Основные команды gdb

- **b метка_или_номер_строки** – установка точки останова на метке или строке с нужным номером
- **x** – запуск программы на выполнение (до точки останова, если она есть, или до конца)
- **s** – шаг со входом – после остановке на точке останова используется для пошагового выполнения программы со входом в подпрограммы
- **n** – шаг с обходом – после остановке на точке останова используется для пошагового выполнения программы без входа в подпрограммы
- **p регистр_или_переменная** – печать значения из регистра или переменной

Кодманда p (print)

- Чтобы вывести значение из регистра с помощью команды `p`, нужно поставить `$` перед названием регистра
- Чтобы вывести значение переменной, нужно преобразовать её к нужному типу аналогично тому, как это делается в Си. Например, если есть переменная `.short` с названием `x`, чтобы её вывести нужно сделать:
`p (short) x`
- Иногда бывает полезен вывод в шестнадцатеричном виде. Для этого используется `p/x`

Дополнительные источники

Дополнительные источники

- https://ru.wikibooks.org/wiki/%D0%90%D1%81%D1%81%D0%B5%D0%BC%D0%B1%D0%BB%D0%B5%D1%80_%D0%B2_Linux_%D0%B4%D0%BB%D1%8F_%D0%BF%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D0%B8%D1%81%D1%82%D0%BE%D0%B2_C – подробное описание синтаксиса AT&T (презентация сделана с него). В самом конце есть немного про gdb.
- <https://ravesli.com/uroki-assemblera/> – неплохой тьюториал по ассемблеру с синтаксисом Intel
- <https://losst.ru/kak-polzovatsya-gdb> – немного подробнее о gdb